

Gendarmerie nationale
Centre Forensique d'Intelligence Artificielle (CFIA)
Leveque Rayan

Scoring de confiance des composants IA

Système IDeXtend

Conformité AI Act — Annexe III §6
Système à haut risque — Application de la loi

Version 0.1 — Brouillon

Table des matières

Introduction	2
1 Détection automatique de langue	3
2 Traduction automatique — NLLB-600M	5
3 Reranker	7
4 OCR — Reconnaissance Optique de Caractères	8
5 ASR — Reconnaissance Automatique de la Parole	10
6 LLM — Grand Modèle de Langage (Qwen3)	12
7 Embeddings — Représentation vectorielle	15
Synthèse et prochaines étapes	16

Introduction

IDeXtend est un système RAG (*Retrieval-Augmented Generation*) d'aide à l'enquête judiciaire déployé au CFIA, Gendarmerie nationale. Il traite des documents hétérogènes (images, audio, vidéo, textes) pour permettre aux enquêteurs d'interroger leur corpus documentaire en langage naturel.

En vertu de l'AI Act (Règlement UE 2024/1689), IDeXtend sera probablement classifié **système à haut risque** au titre de l'Annexe III §6 (application de la loi). Ce dossier documente scientifiquement le **système de scoring de confiance** de chaque brique du pipeline, avec pour chaque composant : la formule mathématique du score, sa justification par l'état de l'art.

État d'implémentation (mai 2026) :

Composant	Modèle	Scoring implémenté	Validé
Détection langue	FastText LID	Oui	Non
Traduction	NLLB-600M	Oui	Non
Reranker	BAAI/bge-reranker-v2-m3	Oui	Non
OCR	QwenVL 2.5-7B	À définir	Non
ASR (Whisper)	whisper-large-v3	À implémenter	Non
LLM (Qwen3)	Qwen3-4B	À implémenter	Non
Embeddings	BAAI/bge-m3	N/A	Non

Chapitre 1

Détection automatique de langue

1.1 Identification

Modèle	facebook/fasttext-language-identification
Nb langues	217
Statut scoring	Implémenté
Fichier	DataPipelines/LanguageDetection.py

1.2 Fonctionnalité

Premier composant de la chaîne : détermine si une traduction est nécessaire. Déclenché sur les 700 premiers + 1500–2500 caractères du texte. Retourne le code langue BCP-47 FLORES-200 (ex. `fra_Latn`, `ara_Arab`).

1.3 Fonction de scoring de confiance

Implémentation dans `LanguageDetection.compute_confidence()`.

FastText applique un softmax sur 217 classes. Le score de confiance est directement la probabilité de la langue prédite :

$$s_{\text{LID}}(x) = P(\hat{l} | x) = \max_{l \in \mathcal{L}} P(l | x) \in (0, 1] \quad (1.1)$$

Les top-2 prédictions sont stockées dans les métadonnées pour auditabilité.

Seuils.

$s \geq 0.90$	Langue certaine	Décision automatique fiable
$0.60 \leq s < 0.90$	Langue probable	Logging, surveiller
$s < 0.60$	Détection ambiguë	Ne pas traduire automatiquement

Métadonnées loguées : `detected_lang`, `top_k` (liste des 2 meilleures prédictions avec scores).

1.4 Justification scientifique

FastText, probabilités softmax [4] ; cas difficiles (code-switching, dialectes) [1].

1.5 Validation empirique

Aucun dataset d'évaluation annoté n'est disponible à ce jour. Le `compute_confidence()` est actuellement validé par inspection manuelle sur des cas réels et des scénarios limites (*smoke testing*).

Métriques à mesurer lors de la constitution du dataset :

- Corrélation entre le score de confiance et la bonne classification ;
- Calibration (ECE) : un score de confiance de 0.8 correspond-il à environ 80 % de réussite sur le dataset ?
- Pouvoir discriminant (AUC-ROC) : le score sépare-t-il les prédictions correctes des incorrectes ?

Statut : *À implémenter.*

Stress tests à conduire. Textes bilingues mélangés, textes très courts (< 20 caractères), arabe dialectal levantin, translittération arabe en latin.

Chapitre 2

Traduction automatique — NLLB-600M

2.1 Identification

Modèle	facebook/nllb-200-distilled-600M
Beam search	$B = 5$ faisceaux
Statut scoring	Implémenté
Fichier	DataPipelines/TranslationModel.py

2.2 Benchmark FLORES+ (200 phrases, 8 paires de langues)

Modèle	FR→EN	EN→FR	FR→AR	AR→FR	ms/phrase
NLLB-600M	41.0	46.3	13.9	29.7	130–180
Qwen3.5-4B	41.2	44.7	14.5	29.3	660–900
TGemma-4B	37.9	40.4	14.5	26.4	620–940

NLLB retenu : gagne 5–6/8 directions, 4–5× plus rapide. Résultats complets dans `scoring_rayan/_archi`

2.3 Fonction de scoring de confiance

Implémentation dans `TranslationModel.compute_confidence()`.

Soit K le nombre de chunks, ℓ_k le log-prob normalisé best-beam, $p_k = \exp(\ell_k)$, et $r_k = \text{output_len}_k / \text{input_len}_k$ le ratio de longueur.

$$\phi = 1 - \frac{|\{k : r_k < 0.3 \text{ ou } r_k > 4\}|}{K} \cdot 0.5 \quad (2.1)$$

$$s_{\text{trad}}(x, \hat{y}) = (0.7 \cdot \bar{p} + 0.3 \cdot \min_k p_k) \cdot \phi \in (0, 1] \quad (2.2)$$

Seuils.

$s \geq 0.70$	Traduction fiable	Exploitable
$0.40 \leq s < 0.70$	Traduction incertaine	Vérification recommandée
$s < 0.40$	Très incertaine	Relecture humaine

Métadonnées loguées : `num_chunks`, `src_lang`, `tgt_lang`, `mean_beam_prob`, `min_beam_prob`, `outlier_chunks`.

2.4 Justification scientifique

NLLB [5] ; calibration beam NMT [6] ; Quality Estimation non supervisée [2].

2.5 Validation empirique

Aucun dataset d'évaluation annoté n'est disponible à ce jour. Le `compute_confidence()` est actuellement validé par inspection manuelle sur des cas réels et des scénarios limites (*smoke testing*).

Métriques à mesurer lors de la constitution du dataset :

- Corrélation entre le score de confiance et la métrique de qualité appropriée (ex. BLEU, chrF++);
- Calibration (ECE) : un score de confiance de 0.8 correspond-il à environ 80 % de réussite sur le dataset ?
- Pouvoir discriminant (AUC-ROC) : le score sépare-t-il les traductions correctes des incorrectes ?

Statut : *À implémenter.*

Chapitre 3

Reranker

3.1 Identification

Modèle	BAAI/bge-reranker-v2-m3 (CrossEncoder)
Statut scoring	Implémenté
Fichier	DataPipelines/Reranker.py

3.2 Fonctionnalité

Le reranker reçoit les 25 chunks candidats des embeddings et les reclasse par pertinence à la question en traitant chaque paire (question, chunk) ensemble. Il filtre les chunks sous le seuil `minimum_acceptable_score = 0.05` selon la méthode de sélection configurée (`smart`, `kmeans`, `top`, `top_kmeans`).

3.3 Fonction de scoring de confiance

Implémentation dans DataPipelines/Reranker.py.

Le cross-encoder produit un logit l_i par paire (question, chunk), converti en score $\tilde{s}_i = \sigma(l_i) \in (0, 1)$.

Le score de confiance actuel est une moyenne pondérée entre la moyenne des chunks retenus et le plus faible d'entre eux :

$$s_{\text{reranker}} = 0.7 \cdot \bar{s} + 0.3 \cdot \min_{i \in \mathcal{I}} \tilde{s}_i \quad (3.1)$$

Cette formule est temporaire. La pondération 0.7/0.3 n'a pas été validée empiriquement. Elle ne tient pas compte de la taille du corpus : avec peu de documents disponibles, le système peut retenir des chunks faiblement pertinents faute de mieux, ce que le score ne reflète pas.

Cas particuliers.

- $|\mathcal{I}| = 0$: $s = 0.0$ (aucun candidat retenu).
- $|\mathcal{I}| = 1$, fast path : $s = 0.5$ (non mesuré par convention).

Seuils.

$s \geq 0.70$	Bons candidats	Réponse LLM probablement pertinente
$0.40 \leq s < 0.70$	Pertinence moyenne	Vérification recommandée
$s < 0.40$	Peu pertinent	Avertissement

Ces seuils sont provisoires et non calibrés.

Métadonnées loguées : `num_selected`, `mean_score`, `min_score`, `max_score`, `per_index_scores`.

Tests de régression automatisés : `Tests/test_reranker_regression.py`.

Chapitre 4

OCR — Reconnaissance Optique de Caractères

4.1 Identification

Champ	Valeur
Modèle actif	QwenVL 2.5-7B (vLLM)
Statut scoring	À implémenter
Fichier	<code>DataPipelines/VLMModel.py</code>

4.2 Fonctionnalité

Le composant OCR convertit les images en texte exploitable. Il est déclenché sur chaque nœud image portant le flag `OCR_FLAG`. Le texte produit est découpé en chunks, vectorisé par BGE-M3, traduit si nécessaire, puis indexé pour la recherche.

Entrée. Image binaire (JPEG, PNG, BMP) $> 100 \times 100$ px, ratio max/min < 10 .

Sortie. Texte brut multi-lignes, ordre de lecture reconstitué.

4.3 Modèles évalués

Modèle	Version	Benchmarké	Retenu
EasyOCR + Tesseract	1.7 + 5.x	Non (legacy)	Non
QwenVL 2.5-7B	7B Q8	Non	Oui
Marker	TDB	Non	Non
Docling (IBM)	TDB	Non	Non
InternVL 3.5-7B	TDB	Non	Non

4.4 Protocole d'évaluation

Métriques. TBD.

Dataset à constituer. TBD

4.5 Fonction de scoring de confiance

4.5.1 Mode legacy — EasyOCR + Tesseract

TBD

4.5.2 Mode VLM / QwenVL (actif) — Proposition

TBD

Seuils provisoires.

Plage	Interprétation	Action
TBD	Confiance moyenne	Relecture recommandée
TBD	Haute confiance	Exploitable
TBD	Faible confiance	Relecture obligatoire

4.6 Justification scientifique

4.7 Validation empirique

Chapitre 5

ASR — Reconnaissance Automatique de la Parole

5.1 Identification

Champ	Valeur
Modèle	openai/whisper-large-v3
Mode	Chunked (30 s/chunk, batch = 8)
Statut scoring	À implémenter
Fichier	DataPipelines/WhisperModel.py

5.2 Fonctionnalité

Whisper transcrit les fichiers audio et vidéo en texte. Dans le contexte judiciaire : écoutes téléphoniques, auditions, vidéos de surveillance.

5.3 Modèles évalués

Modèle	Version	Benchmarké	Retenu
whisper-large-v3	large-v3	Non	Oui
whisper-medium	medium	Non	Non
Faster-Whisper CT2	CT2 large-v3	Non	Non

5.4 Fonction de scoring de confiance

`WhisperModel.infer()` retourne uniquement `result["text"]` — aucun beam score ni métadonnée de confiance n'est exposé dans l'état actuel.

Pour implémenter un scoring, il faudrait modifier `infer()` afin de récupérer les `output_scores` de la pipeline HuggingFace. Une approche cohérente avec `TranslationModel` serait d'utiliser les log-probs best-beam par chunk (même architecture seq2seq, même beam search). Cette piste reste à valider — aucune formule n'est fixée à ce stade.

5.5 Justification scientifique

Architecture Whisper [7].

5.6 Validation empirique

Aucun dataset d'évaluation annoté n'est disponible à ce jour. Le `compute_confidence()` est actuellement validé par inspection manuelle sur des cas réels et des scénarios limites (*smoke testing*).

Métriques à mesurer lors de la constitution du dataset :

- Corrélation entre le score de confiance et la métrique de qualité appropriée (ici $1 - \text{WER}$) ;
- Calibration (ECE) : un score de confiance de 0.8 correspond-il à environ 80 % de réussite sur le dataset ?
- Pouvoir discriminant (AUC-ROC) : le score sépare-t-il les transcriptions correctes des incorrectes ?

Statut : *À implémenter.*

Chapitre 6

LLM — Grand Modèle de Langage (Qwen3)

6.1 Identification

Config active	Qwen3-4B-Instruct-ctx-8k-30-user
Modèle	Qwen/Qwen3-4B
Backend	vLLM
Contexte	8 000 tokens
Statut scoring	À implémenter
Fichier	DataPipelines/LlmModel.py

6.2 Fonctionnalité

Le LLM est le composant de génération finale : réponses aux enquêteurs, résumés, extraction d'entités, graphe de connaissances.

6.3 Pourquoi les log-probs seuls sont insuffisants

Pour un LLM instruit par RLHF/DPO, la probabilité de séquence $P(\hat{y}|x) = \prod_t P(y_t|y_{<t}, x)$ est :

1. **Mal calibrée** post-RLHF (scores absolus peu interprétables).
2. **Biaisée par la longueur** : les réponses courtes ont mécaniquement des log-probs plus élevés.
3. **Non discriminante des hallucinations** : un modèle peut être très confiant en générant une information fausse.

Le score de confiance du LLM ne doit pas être interprété comme une probabilité de vérité. Il est utilisé comme un signal de triage pour détecter les réponses nécessitant une relecture humaine.

6.4 Approche de scoring proposée

Signal 1 — *Retrieval quality*. On mesure si les chunks fournis au LLM contiennent les éléments nécessaires pour répondre à la question.

$$s_{\text{retrieval}}(q, C) = \mathbb{1}[\text{gold chunk} \in C_{1:k}] \quad (6.1)$$

En absence de dataset annoté, ce signal est estimé par inspection humaine sur un échantillon de questions.

Signal 2 — *Groundedness*. La réponse est découpée en affirmations atomiques. Chaque affirmation est classée comme *supported*, *unsupported* ou *contradicted* par les chunks fournis.

$$s_{\text{groundedness}}(\hat{y}, C) = \frac{\# \text{supported claims}}{\# \text{total claims}} \quad (6.2)$$

$$r_{\text{contradiction}}(\hat{y}, C) = \frac{\# \text{contradicted claims}}{\# \text{total claims}} \quad (6.3)$$

Signal 3 — *Abstention*. Le système doit refuser de conclure lorsque les documents ne contiennent pas l'information demandée.

$$s_{\text{abstain}} = \mathbb{K}[\text{réponse absente des documents} \Rightarrow \text{abstention ou réponse qualifiée}] \quad (6.4)$$

Score de triage.

$$s_{\text{triage}} = \min\left(1, \alpha s_{\text{groundedness}} + \beta s_{\text{retrieval}} - \lambda r_{\text{contradiction}}\right) \cdot s_{\text{abstain}} \quad (6.5)$$

avec $\alpha + \beta = 1$, $\lambda \geq 0$. Le facteur $s_{\text{abstain}} \in \{0, 1\}$ agit comme un veto : si le système devait s'abstenir et ne l'a pas fait, le score est nul. Les coefficients α, β, λ ne sont pas fixés a priori ; ils doivent être calibrés sur le dataset de validation [3].

Règle opérationnelle.

Condition	Interprétation	Action
Preuve absente	Réponse <i>ungrounded</i>	<i>Abstention</i> obligatoire
Contradiction détectée	Risque élevé	Relecture humaine
$s_{\text{groundedness}} < 0.8$	<i>Grounding</i> insuffisant	Relecture humaine
Toutes les <i>claims</i> supportées	Réponse exploitable	Citation obligatoire

6.5 Validation empirique

Dataset cible

Affaires	~10 cas réels anonymisés
Documents	~50 documents au total
Questions annotées	schéma ci-dessous

```
{
  "case_id":      "affaire_001",
  "question":     "Quel véhicule était impliqué ?",
  "reference_answer": "Un Renault Clio immatriculé AB-123-CD",
  "golden_chunks": ["chunk_002", "chunk_006"],
  "answerable":   true,
  "ambiguous":    false
}
```

Le dataset doit être constitué et verrouillé avant tout réglage des seuils et coefficients.

Champs calculés à l'évaluation (non dans le dataset statique) : `retrieved_chunks`, `generated_answer`, `abstained`, `human_score` (jugement humain 0–1, proxy pour $s_{\text{groundedness}}$ en l'absence d'annotation *claim-level*).

Métriques

- **Retrieval hit@k** : le chunk annoté pertinent est-il dans les k chunks fournis au LLM ?
- **Exactitude de réponse** : réponse correcte, partielle, incorrecte ou abstention (jugement humain).
- ***Unsupported claims rate*** : proportion d'affirmations absentes des sources.
- ***Contradiction rate*** : proportion d'affirmations contredites par les sources.
- ***Abstention correcte*** : le système refuse-t-il de conclure sur les questions négatives et ambiguës ?

Statut actuel : validation manuelle exploratoire uniquement. Le système ne doit pas être présenté comme calibré tant que ce dataset n'a pas été constitué.

Chapitre 7

Embeddings — Représentation vectorielle

7.1 Identification

Modèle	BAAI/bge-m3
Dimension	1 024
Statut scoring	À implémenter
Fichier	DataPipelines/EmbeddingsModel.py

7.2 Fonctionnalité

Le modèle d’embeddings encode chunks et questions en vecteurs denses. La documentation du benchmark de sélection est dans `documentation_pierre/choosing_embeddings.md`.

7.3 Benchmark (extrait)

Modèle	Position moy. réponse	Retenu
paraphrase-multilingual-MiniLM-L12-v2	31	Non
BAAI/bge-m3	4.7	Oui

Score = rang moyen de la réponse correcte (plus bas = mieux). Protocole « needle in the haystack », 4 langues, script `evaluate_embeddings.py`.

7.4 Scoring de confiance

`EmbeddingsModel` est un encodeur pur : il produit un vecteur, sans score de confiance associé. Le retrieval est effectué en aval par `CypherHandler.search_embeddings()` dans `LargeLanguageHandler/rag.py`.

Un `compute_confidence()` au niveau de l’encodeur n’a pas de sens. La qualité du retrieval est évaluée via le benchmark existant (position moyenne de la réponse correcte, script `evaluate_embeddings.py`) et via la métrique **retrieval hit@k** du dataset d’évaluation LLM (chapitre 5).

Synthèse et prochaines étapes

Trois composants ont déjà leur `compute_confidence()` implémentée : Reranker, Traduction (NLLB) et Détection de langue (FastText). Les quatre composants restants nécessitent une implémentation, dont le LLM qui requiert une approche composite (ancrage contextuel + confiance verbalisée).

Actions prioritaires.

1. Constituer les datasets d'évaluation annotés (section 4 de chaque fiche).
2. Implémenter `compute_confidence()` pour OCR, ASR, Embeddings et LLM.
3. Conduire les expériences de validation empirique : Spearman ρ , ECE, AUC-ROC.
4. Compléter le benchmark OCR : Marker et Docling vs QwenVL 2.5-7B.

Limite structurelle : explicabilité et résistance au re-benchmark. IDeXtend est conçu pour être modulaire : chaque composant est remplaçable indépendamment. Cette modularité est un atout technique, mais elle crée une tension avec les exigences d'explicabilité de l'AI Act.

Valider un composant nécessite un dataset annoté, du temps d'annotation, et une infrastructure d'évaluation. En pratique, lorsqu'un utilisateur ou un opérateur souhaite substituer un modèle par une version plus récente — même si le remplacement est techniquement transparent (*plug and play*) — la résistance métier et terrain à relancer un benchmark complet est réelle. Les raisons sont multiples : coût d'annotation, indisponibilité des données de test originales, pression opérationnelle, sentiment que “le nouveau modèle est forcément meilleur”.

Ce risque est structurel : un système peut dériver silencieusement dès lors qu'un composant est mis à jour sans re-validation. L'AI Act (Art. 9 et 15) exige pourtant une réévaluation à chaque changement substantiel.

Piste. Définir contractuellement, pour chaque composant, un *smoke test* minimal automatisé (quelques cas annotés en dur dans le dépôt) qui tourne à chaque mise à jour de modèle en CI/CD. Ce n'est pas une validation complète, mais c'est une barrière minimale contre les régressions silencieuses et un signal d'alerte si le comportement change.

Bibliographie

- [1] Isaac Caswell et al. Language ID in the wild. In *COLING 2020*, 2020.
- [2] Marina Fomicheva et al. Unsupervised quality estimation for neural machine translation. *TACL*, 8 :539–555, 2020.
- [3] Chuan Guo et al. On calibration of modern neural networks. In *ICML 2017*, 2017.
- [4] Armand Joulin et al. Bag of tricks for efficient text classification. *arXiv :1607.01759*, 2016.
- [5] NLLB Team et al. No language left behind. *arXiv :2207.04672*, 2022.
- [6] Myle Ott et al. Analyzing uncertainty in neural machine translation. In *ICML 2018*, 2018.
- [7] Alec Radford et al. Robust speech recognition via large-scale weak supervision. *arXiv :2212.04356*, 2022.